

▼ KNN Classification on the Breast Cancer Dataset

This notebook builds a **K-Nearest Neighbors (KNN)** classifier that predicts whether a breast tumor is **malignant or benign** based on diagnostic measurements.

Workflow

- 1. Load and inspect the data
- 2. Prepare and clean the dataset
- 3. Explore the data visually
- 4. Split and scale the features
- 5. Tune the hyperparameter **K** using cross-validation
- 6. Train the final model and evaluate it (accuracy, confusion matrix, ROC-AUC)
- 7. Summarize findings

```
import numpy as np
import pandas as pd
```

```
df = pd.read_csv('/content/drive/MyDrive/breast-cancer.csv')
df.head()
```

	id	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean
0	842302	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001
1	842517	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869
2	84300903	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974
3	84348301	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414
4	84358402	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980

5 rows × 32 columns

```
df.shape
```

(569, 32)

```
df.drop(columns=['id'],inplace=True)
df.head()
```

	diagnosis	radius_mean	texture_mean	perimeter_mean	area_mean	smoothness_mean	compactness_mean	concavity_mean	concave points_mean
0	M	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.1471
1	M	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.0706
2	M	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.1279
3	M	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.1052
4	M	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.1043

5 rows × 31 columns

```
from sklearn.model_selection import train_test_split
X_train, X_test, y_train, y_test = train_test_split(df.iloc[:,1:], df.iloc[:,0],test_size=0.2, random_state=2)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
from sklearn.neighbors import KNeighborsClassifier

knn = KNeighborsClassifier(n_neighbors=3)
```

```
knn.fit(X_train,y_train)
```

```
▼ KNeighborsClassifier ⓘ ?
KNeighborsClassifier(n_neighbors=3)
```

```
from sklearn.metrics import accuracy_score

y_pred = knn.predict(X_test)

accuracy_score(y_test, y_pred)
```

```
0.9912280701754386
```

```
scores = []

for i in range(1,16):

    knn = KNeighborsClassifier(n_neighbors=i)

    knn.fit(X_train,y_train)

    y_pred = knn.predict(X_test)

    scores.append(accuracy_score(y_test, y_pred))

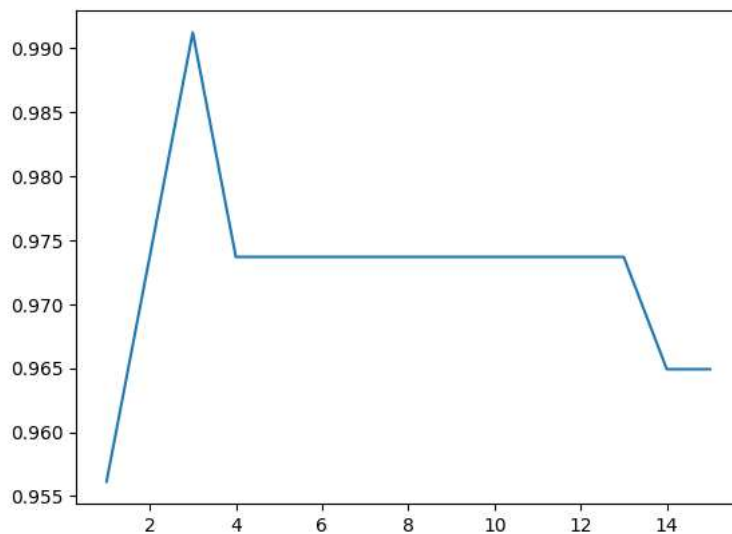
    print(accuracy_score(y_test, y_pred))
```

```
0.956140350877193
0.9736842105263158
0.9912280701754386
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9736842105263158
0.9649122807017544
0.9649122807017544
```

```
import matplotlib.pyplot as plt

plt.plot(range(1,16),scores)
```

```
[<matplotlib.lines.Line2D at 0x7b7677e865d0>]
```



Save Model

```
from sklearn.pipeline import Pipeline
from sklearn.preprocessing import StandardScaler
from sklearn.neighbors import KNeighborsClassifier
from joblib import dump

# Create final KNN model
knn_pipeline = Pipeline([
    ('scaler', StandardScaler()),
    ('knn', KNeighborsClassifier(n_neighbors=3))
])

# Train the complete pipeline
knn_pipeline.fit(X_train, y_train)

# Save model
dump(knn_pipeline, "KNNModel.joblib")

print("KNN model saved successfully!")

KNN model saved successfully!
```

Flask Code

```
from flask import Flask, render_template, request
from joblib import load
import numpy as np

app = Flask(__name__)

# Load trained KNN pipeline
model = load("KNNModel.joblib")

@app.route('/')
def home():
    return render_template("index.html")

@app.route('/predict', methods=['POST'])
def predict():

    # -----
    # Mean Features
    # -----
    radius_mean = float(request.form['radius_mean'])
    texture_mean = float(request.form['texture_mean'])
    perimeter_mean = float(request.form['perimeter_mean'])
    area_mean = float(request.form['area_mean'])
    smoothness_mean = float(request.form['smoothness_mean'])
    compactness_mean = float(request.form['compactness_mean'])
    concavity_mean = float(request.form['concavity_mean'])
    concave_points_mean = float(request.form['concave_points_mean'])
    symmetry_mean = float(request.form['symmetry_mean'])
    fractal_dimension_mean = float(request.form['fractal_dimension_mean'])

    # -----
    # Standard Error Features
    # -----
    radius_se = float(request.form['radius_se'])
    texture_se = float(request.form['texture_se'])
    perimeter_se = float(request.form['perimeter_se'])
    area_se = float(request.form['area_se'])
    smoothness_se = float(request.form['smoothness_se'])
    compactness_se = float(request.form['compactness_se'])
    concavity_se = float(request.form['concavity_se'])
    concave_points_se = float(request.form['concave_points_se'])
    symmetry_se = float(request.form['symmetry_se'])
    fractal_dimension_se = float(request.form['fractal_dimension_se'])

    # -----
    # Worst Features
    # -----
```

```

#
radius_worst = float(request.form['radius_worst'])
texture_worst = float(request.form['texture_worst'])
perimeter_worst = float(request.form['perimeter_worst'])
area_worst = float(request.form['area_worst'])
smoothness_worst = float(request.form['smoothness_worst'])
compactness_worst = float(request.form['compactness_worst'])
concavity_worst = float(request.form['concavity_worst'])
concave_points_worst = float(request.form['concave_points_worst'])
symmetry_worst = float(request.form['symmetry_worst'])
fractal_dimension_worst = float(request.form['fractal_dimension_worst'])

# -----
# Create input array
# IMPORTANT:
# Feature order must match
# training data
# -----
input_data = np.array([[
    radius_mean,
    texture_mean,
    perimeter_mean,
    area_mean,
    smoothness_mean,
    compactness_mean,
    concavity_mean,
    concave_points_mean,
    symmetry_mean,
    fractal_dimension_mean,

    radius_se,
    texture_se,
    perimeter_se,
    area_se,
    smoothness_se,
    compactness_se,
    concavity_se,
    concave_points_se,
    symmetry_se,
    fractal_dimension_se,

    radius_worst,
    texture_worst,
    perimeter_worst,
    area_worst,
    smoothness_worst,
    compactness_worst,
    concavity_worst,
    concave_points_worst,
    symmetry_worst,
    fractal_dimension_worst
]])

# -----
# Prediction
# -----
prediction = model.predict(input_data)

# -----
# Convert prediction to result
# -----
if prediction[0] == 'M':
    result = "Malignant"
elif prediction[0] == 'B':
    result = "Benign"
elif prediction[0] == 1:
    result = "Malignant"
else:
    result = "Benign"

return render_template(
    "index.html",
    prediction_text=f"Prediction : {result}"
)

if __name__ == "__main__":

```

```
app.run(debug=True)
```

```
* Serving Flask app '__main__'
* Debug mode: on
INFO:werkzeug:WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server inst
* Running on http://127.0.0.1:5000
INFO:werkzeug:Press CTRL+C to quit
INFO:werkzeug: * Restarting with watchdog (inotify)
```

HTML Code :-

```
<!DOCTYPE html>
<html lang="en">

<head>

  <meta charset="UTF-8">

  <meta name="viewport" content="width=device-width, initial-scale=1.0">

  <title>KNN Breast Cancer Classification</title>

  <style>

    * {
      box-sizing: border-box;
    }

    body {
      margin: 0;
      font-family: Arial, sans-serif;
      background: #f4f7fb;
      color: #333;
    }

    .container {
      width: 95%;
      max-width: 1100px;
      margin: 30px auto;
    }

    .card {
      background: white;
      padding: 30px;
      border-radius: 12px;
      box-shadow: 0 4px 15px rgba(0, 0, 0, 0.08);
    }

    h1 {
      text-align: center;
      color: #1769aa;
      margin-bottom: 8px;
    }

    .subtitle {
      text-align: center;
      color: #666;
      margin-bottom: 10px;
    }

    .instruction {
      text-align: center;
      color: #777;
      font-size: 13px;
      margin-bottom: 25px;
    }

    .section-title {
      background: #eaf3fb;
      color: #1769aa;
      padding: 12px 15px;
      border-radius: 6px;
      margin-top: 25px;
      margin-bottom: 15px;
      font-size: 18px;
      font-weight: bold;
    }
```

```

}

.section-note {
  font-size: 13px;
  color: #777;
  margin-bottom: 12px;
}

.input-grid {
  display: grid;
  grid-template-columns: repeat(2, 1fr);
  gap: 15px;
}

.input-group {
  display: flex;
  flex-direction: column;
}

label {
  font-size: 14px;
  margin-bottom: 6px;
  font-weight: bold;
}

input {
  padding: 10px;
  border: 1px solid #ccc;
  border-radius: 5px;
  font-size: 14px;
}

input:focus {
  outline: none;
  border-color: #1769aa;
}

input::placeholder {
  color: #999;
  font-size: 12px;
}

.button-area {
  text-align: center;
  margin-top: 30px;
}

button {
  background: #1769aa;
  color: white;
  border: none;
  padding: 13px 35px;
  border-radius: 6px;
  font-size: 16px;
  cursor: pointer;
}

button:hover {
  background: #0d568d;
}

.result {
  margin-top: 25px;
  padding: 15px;
  text-align: center;
  background: #e8f7ed;
  color: #218838;
  border-radius: 6px;
  font-size: 18px;
  font-weight: bold;
}

.footer {
  text-align: center;
  color: #777;
  margin-top: 20px;
  font-size: 13px;
}

```

```

    }

    @media(max-width: 700px) {

        .input-grid {
            grid-template-columns: 1fr;
        }

        .card {
            padding: 20px;
        }

    }

</style>
</head>

<body>

<div class="container">

    <div class="card">

        <h1>KNN Breast Cancer Classification</h1>

        <div class="subtitle">
            Classification using K-Nearest Neighbors
        </div>

        <div class="instruction">
            Enter the 30 numerical features below.
        </div>

        <form action="/predict" method="POST">

            <!-- 1. MEAN FEATURES -->

            <div class="section-title">
                1. Mean Features
            </div>

            <div class="section-note">
                Example range: values commonly vary approximately from small decimal
                values to larger measurements depending on the feature.
            </div>

            <div class="input-grid">

                <div class="input-group">
                    <label>Radius Mean</label>
                    <input type="number"
                        step="any"
                        name="radius_mean"
                        placeholder="Example: 14.13"
                        min="0"
                        required>
                </div>

                <div class="input-group">
                    <label>Texture Mean</label>
                    <input type="number"
                        step="any"
                        name="texture_mean"
                        placeholder="Example: 19.26"
                        min="0"
                        required>
                </div>

                <div class="input-group">
                    <label>Perimeter Mean</label>

```

```

</label>Perimeter Mean</label>
<input type="number"
      step="any"
      name="perimeter_mean"
      placeholder="Example: 91.97"
      min="0"
      required>
</div>

<div class="input-group">
  <label>Area Mean</label>
  <input type="number"
        step="any"
        name="area_mean"
        placeholder="Example: 654.9"
        min="0"
        required>
</div>

<div class="input-group">
  <label>Smoothness Mean</label>
  <input type="number"
        step="any"
        name="smoothness_mean"
        placeholder="Example: 0.09636"
        min="0"
        required>
</div>

<div class="input-group">
  <label>Compactness Mean</label>
  <input type="number"
        step="any"
        name="compactness_mean"
        placeholder="Example: 0.1043"
        min="0"
        required>
</div>

<div class="input-group">
  <label>Concavity Mean</label>
  <input type="number"
        step="any"
        name="concavity_mean"
        placeholder="Example: 0.0889"
        min="0"
        required>
</div>

<div class="input-group">
  <label>Concave Points Mean</label>
  <input type="number"
        step="any"
        name="concave_points_mean"
        placeholder="Example: 0.048"
        min="0"
        required>
</div>

<div class="input-group">
  <label>Symmetry Mean</label>
  <input type="number"
        step="any"
        name="symmetry_mean"
        placeholder="Example: 0.181"
        min="0"
        required>
</div>

<div class="input-group">
  <label>Fractal Dimension Mean</label>

```

```

        <input type="number"
              step="any"
              name="fractal_dimension_mean"
              placeholder="Example: 0.062"
              min="0"
              required>
    </div>

</div>

<!-- 2. STANDARD ERROR FEATURES -->

<div class="section-title">
    2. Standard Error Features
</div>

<div class="section-note">
    Example values are shown as a guide. Enter numerical values
    corresponding to the same feature definitions used during training.
</div>

<div class="input-grid">

    <div class="input-group">
        <label>Radius SE</label>
        <input type="number"
              step="any"
              name="radius_se"
              placeholder="Example: 0.405"
              min="0"
              required>
    </div>

    <div class="input-group">
        <label>Texture SE</label>
        <input type="number"
              step="any"
              name="texture_se"
              placeholder="Example: 1.216"
              min="0"
              required>
    </div>

    <div class="input-group">
        <label>Perimeter SE</label>
        <input type="number"
              step="any"
              name="perimeter_se"
              placeholder="Example: 2.866"
              min="0"
              required>
    </div>

    <div class="input-group">
        <label>Area SE</label>
        <input type="number"
              step="any"
              name="area_se"
              placeholder="Example: 40.51"
              min="0"
              required>
    </div>

    <div class="input-group">
        <label>Smoothness SE</label>
        <input type="number"
              step="any"
              name="smoothness_se"
              placeholder="Example: 0.007"
              min="0"
    
```

```

        required>
    </div>

    <div class="input-group">
        <label>Compactness SE</label>
        <input type="number"
            step="any"
            name="compactness_se"
            placeholder="Example: 0.021"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Concavity SE</label>
        <input type="number"
            step="any"
            name="concavity_se"
            placeholder="Example: 0.026"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Concave Points SE</label>
        <input type="number"
            step="any"
            name="concave_points_se"
            placeholder="Example: 0.012"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Symmetry SE</label>
        <input type="number"
            step="any"
            name="symmetry_se"
            placeholder="Example: 0.021"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Fractal Dimension SE</label>
        <input type="number"
            step="any"
            name="fractal_dimension_se"
            placeholder="Example: 0.003"
            min="0"
            required>
    </div>
</div>

<!-- 3. WORST FEATURES -->

<div class="section-title">
    3. Worst Features
</div>

<div class="section-note">
    These represent the worst/largest values calculated from the
    cell-nucleus measurements.
</div>

<div class="input-grid">

    <div class="input-group">
        <label>Radius Worst</label>
        <input type="number"

```

```
        step="any"
        name="radius_worst"
        placeholder="Example: 17.33"
        min="0"
        required>
    </div>

    <div class="input-group">
        <label>Texture Worst</label>
        <input type="number"
            step="any"
            name="texture_worst"
            placeholder="Example: 25.0"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Perimeter Worst</label>
        <input type="number"
            step="any"
            name="perimeter_worst"
            placeholder="Example: 110.2"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Area Worst</label>
        <input type="number"
            step="any"
            name="area_worst"
            placeholder="Example: 900.0"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Smoothness Worst</label>
        <input type="number"
            step="any"
            name="smoothness_worst"
            placeholder="Example: 0.12"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Compactness Worst</label>
        <input type="number"
            step="any"
            name="compactness_worst"
            placeholder="Example: 0.25"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Concavity Worst</label>
        <input type="number"
            step="any"
            name="concavity_worst"
            placeholder="Example: 0.30"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Concave Points Worst</label>
        <input type="number"
```

```

        step="any"
        name="concave_points_worst"
        placeholder="Example: 0.15"
        min="0"
        required>
    </div>

    <div class="input-group">
        <label>Symmetry Worst</label>
        <input type="number"
            step="any"
            name="symmetry_worst"
            placeholder="Example: 0.29"
            min="0"
            required>
    </div>

    <div class="input-group">
        <label>Fractal Dimension Worst</label>
        <input type="number"
            step="any"
            name="fractal_dimension_worst"
            placeholder="Example: 0.08"
            min="0"
            required>
    </div>
</div>

<!-- PREDICT BUTTON -->
<div class="button-area">
    <button type="submit">
        Predict
    </button>
</div>

</form>

<!-- RESULT -->
{% if prediction_text %}
<div class="result">
    {{ prediction_text }}
</div>
{% endif %}

</div>

<div class="footer">
    A Project by Dr. Sachin Balawant Takmare |
    K-Nearest Neighbors Classifier
</div>
</div>
</body>
</html>

```

File ["/tmp/ipykernel_2142/1751423095.py"](#), line 27

max-width: 1100px;

^

SyntaxError: invalid decimal literal

Next steps: [Explain error](#)